First-order approximation algorithms

Weekend 1, Friday morning

Carlos Cotrini

Friday 4 September 2026

First-order approximation algorithms

Weekend 1, Friday morning

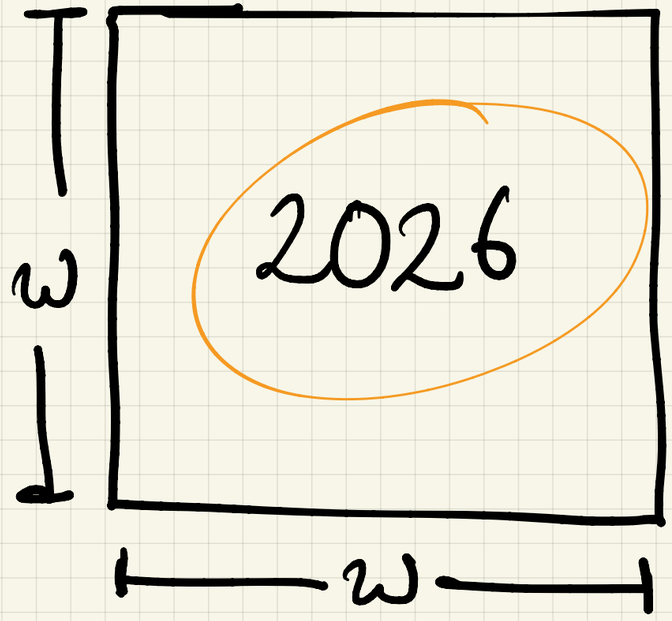
Carlos Cotrini

Friday 4 September 2026

1

Section 1

Two problems, done by hand

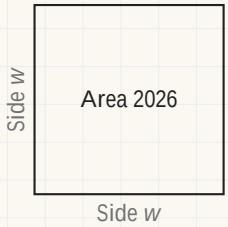


w	estimate w^2	diff: 2026 − w^2 / target − estimate	Action
45	2025	+1	↑
45.5	2070.25	−44.75	↓
45.05	2029.5	−3.5	↓

$w * w = 2026$

Heron

Calculating square roots

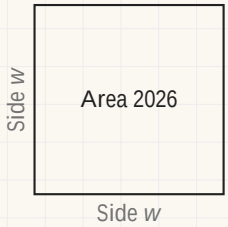


How long is the side?

Find w with $w^2 = 2026$.

w	w^2	$l = 2026 - w^2$	Update

Calculating square roots

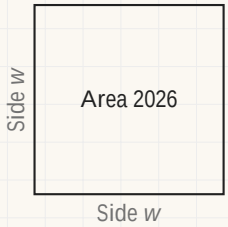


How long is the side?

Find w with $w^2 = 2026$.

w	w^2	$l = 2026 - w^2$	Update
40	1600	$+426$	↑

Calculating square roots

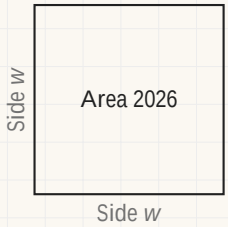


How long is the side?

Find w with $w^2 = 2026$.

w	w^2	$I = 2026 - w^2$	Update
40	1600	$+426$	$\uparrow$
50	2500	-474	$\downarrow$

Calculating square roots

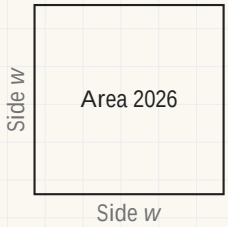


How long is the side?

Find w with $w^2 = 2026$.

w	w^2	$I = 2026 - w^2$	Update
40	1600	$+426$	$\uparrow$
50	2500	-474	$\downarrow$
45	2025	$+1$	$\uparrow$

Calculating square roots

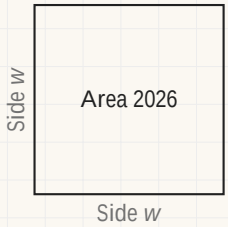


How long is the side?

Find w with $w^2 = 2026$.

w	w^2	$l = 2026 - w^2$	Update
40	1600	$+426$	↑
50	2500	-474	↓
45	2025	$+1$	↑
⋮	⋮	⋮	

Calculating square roots



How long is the side?

Find w with $w^2 = 2026$.

w	w^2	$l = 2026 - w^2$	Update
40	1600	$+426$	↑
50	2500	-474	↓
45	2025	$+1$	↑
⋮	⋮	⋮	

Nobody here computed a square root. Each row is a guess that was scored, and the score said which way to move.

Solving an equation

Find w with $w^3 - 3w^2 + 3w - 2 = 4$.

estimate

target

w	$f(w)$	$l = 4 - f(w)$	Update
5	63	4-63=-59	↓
3			

Solving an equation

Find w with $w^3 - 3w^2 + 3w - 2 = 4$.

w	$f(w)$	$I = 4 - f(w)$	Update
0	-2	$+6$	$\uparrow$

Solving an equation

Find w with $w^3 - 3w^2 + 3w - 2 = 4$.

w	$f(w)$	$I = 4 - f(w)$	Update
0	-2	$+6$	$\uparrow$
6	124	-120	$\downarrow$

Solving an equation

Find w with $w^3 - 3w^2 + 3w - 2 = 4$.

w	$f(w)$	$l = 4 - f(w)$	Update
0	-2	$+6$	$\uparrow$
6	124	-120	$\downarrow$
2	0	$+4$	$\uparrow$

Solving an equation

Find w with $w^3 - 3w^2 + 3w - 2 = 4$.

w	$f(w)$	$l = 4 - f(w)$	Update
0	-2	$+6$	$\uparrow$
6	124	-120	$\downarrow$
2	0	$+4$	$\uparrow$
$\vdots$	$\vdots$	$\vdots$	

Solving an equation

Find w with $w^3 - 3w^2 + 3w - 2 = 4$.

w	$f(w)$	$l = 4 - f(w)$	Update
0	-2	$+6$	$\uparrow$
6	124	-120	$\downarrow$
2	0	$+4$	$\uparrow$
$\vdots$	$\vdots$	$\vdots$	

The same four columns, and only f changed.

Written another way,

$$w^3 - 3w^2 + 3w - 2 = (w-1)^3 - 1,$$

so the equation asks for $(w-1)^3 = 5$: a cube root wearing a cubic.

```
def solve( f , target)
    w ← "guess"
    while |l| > 0.0001:
        l ← target − f(w)
        if l>0 :
            w ← w + 0.001 l
        else:
            w ← w − 0.001 l
    return w
```

Goal

Find w

$f(w) = target$

```
def solve( f , target, α)
    w ← "guess"
    while |l| > 0.0001:
        l ← target − f(w)
        w ← w + α l
    return w
```

Goal

Find w

$f(w) = target$

Robbins–Monro algorithm

The algorithm

```
def solve(f, target, w, alpha):
    l = target - f(w)           # the verdict: how far off, and which way
    while abs(l) > 0.001:       # not close enough yet
        w = w + alpha * l       # the update: move that way
        l = target - f(w)       # score the new guess
    return w
```

The algorithm

```
def solve(f, target, w, alpha):
    l = target - f(w)          # the verdict: how far off, and which way
    while abs(l) > 0.001:       # not close enough yet
        w = w + alpha * l       # the update: move that way
        l = target - f(w)       # score the new guess
    return w
```

The two problems of the last ten minutes are the same call, with a different `f`:

```
def square(w):
    return w**2
def cubic(w):
    return w**3 - 3*w**2 + 3*w - 2
solve(square, 2026, 40, 0.005)
solve(cubic,     4,  0, 0.05)
```

The algorithm

```
def solve(f, target, w, alpha):
    l = target - f(w)          # the verdict: how far off, and which way
    while abs(l) > 0.001:      # not close enough yet
        w = w + alpha * l      # the update: move that way
        l = target - f(w)      # score the new guess
    return w
```

The two problems of the last ten minutes are the same call, with a different `f`:

```
def square(w):
    return w**2
def cubic(w):
    return w**3 - 3*w**2 + 3*w - 2
solve(square, 2026, 40, 0.005)
solve(cubic,     4,  0, 0.05)
```

Everything on this slide is settled except `alpha`. That is the next section.

2

Section 2

Iterative problem solving

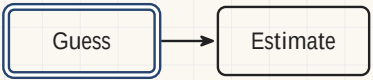
Iterative problem solving

Iterative problem solving

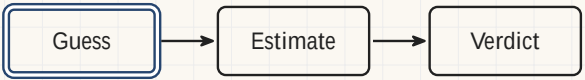
Guess A value of w, however crude.

Iterative problem solving

Guess	A value of w, however crude.
Estimate	What the guess predicts.

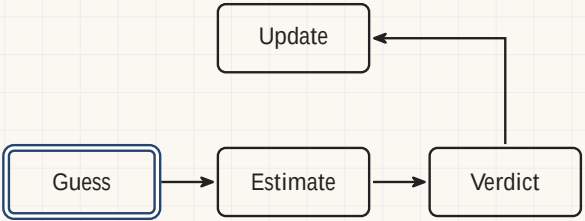


Iterative problem solving



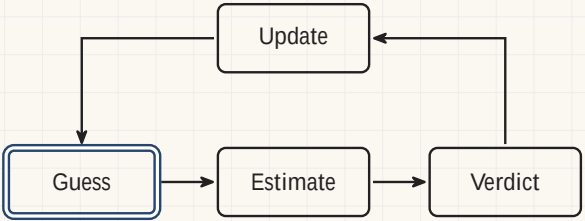
Guess	A value of w, however crude.
Estimate	What the guess predicts.
Verdict	The signed miss I.

Iterative problem solving



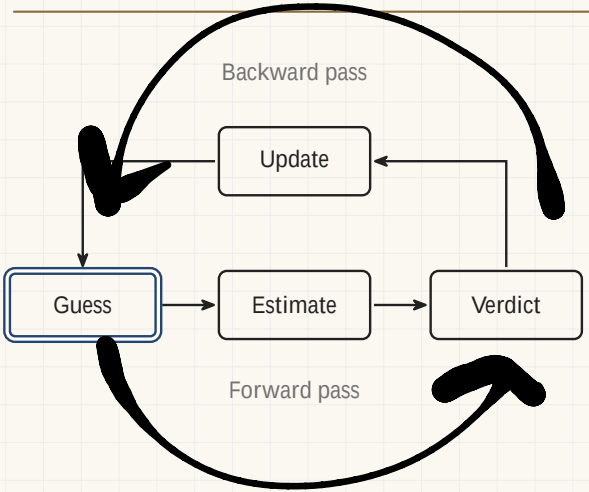
Guess	A value of w, however crude.
Estimate	What the guess predicts.
Verdict	The signed miss l.
Update	Move w the way the verdict says.

Iterative problem solving



Guess	A value of w, however crude.
Estimate	What the guess predicts.
Verdict	The signed miss l.
Update	Move w the way the verdict says.

Iterative problem solving



Guess	A value of w, however crude.
Estimate	What the guess predicts.
Verdict	The signed miss l.
Update	Move w the way the verdict says.

The update rule

$$w := w$$

w	w^2	I

w	w^2	I

The update rule

$$w := w + \underbrace{l}_{\text{The verdict}}$$

w	w^2	l

w	w^2	l

The update rule

$$w := w + \underbrace{\alpha}_{\text{How far to move}} \underbrace{l}_{\text{The verdict}}$$

w	w^2	l

w	w^2	l

The update rule

$$w := w + \underbrace{\alpha}_{\text{How far to move}} \underbrace{l}_{\text{The verdict}}$$

w	w^2	l
40.000	1600.000	426.000
42.130	1774.937	251.063
43.385	1882.286	143.714
44.104	1945.153	80.847
44.508	1980.973	45.027
44.733	2001.064	24.936
44.858	2012.234	13.766
44.927	2018.414	7.586
44.965	2021.824	4.176
44.986	2023.702	2.298
44.997	2024.736	1.264
45.003	2025.305	0.695

w	w^2	l

- Starting at $w = 40$, with $\alpha = 0.005$.

The update rule

$$w := w + \underbrace{\alpha}_{\text{How far to move}} \underbrace{l}_{\text{The verdict}}$$

w	w^2	l
40.000	1600.000	426.000
42.130	1774.937	251.063
43.385	1882.286	143.714
44.104	1945.153	80.847
44.508	1980.973	45.027
44.733	2001.064	24.936
44.858	2012.234	13.766
44.927	2018.414	7.586
44.965	2021.824	4.176
44.986	2023.702	2.298
44.997	2024.736	1.264
45.003	2025.305	0.695

w	w^2	l
45.007	2025.618	0.382
45.009	2025.790	0.210
45.010	2025.884	0.116
45.010	2025.936	0.064
45.011	2025.965	0.035
45.011	2025.981	0.019
45.011	2025.989	0.011
45.011	2025.994	0.006
45.011	2025.997	0.003
45.011	2025.998	0.002
45.011	2025.999	0.001

- Starting at $w = 40$, with $\alpha = 0.005$.
- The whole run, all 23 rounds of it.
- From the □fth row of this block, w has stopped moving and the verdict is still shrinking.

The update rule

$$w := w + \underbrace{\alpha}_{\text{How far to move}} \underbrace{I}_{\text{The verdict}}$$

w	w^2	I
40.000	1600.000	426.000
42.130	1774.937	251.063
43.385	1882.286	143.714
44.104	1945.153	80.847
44.508	1980.973	45.027
44.733	2001.064	24.936
44.858	2012.234	13.766
44.927	2018.414	7.586
44.965	2021.824	4.176
44.986	2023.702	2.298
44.997	2024.736	1.264
45.003	2025.305	0.695

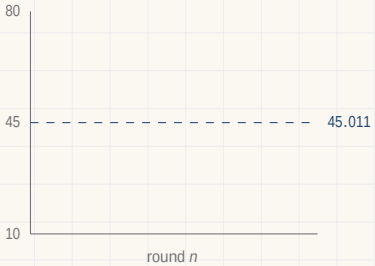
w	w^2	I
45.007	2025.618	0.382
45.009	2025.790	0.210
45.010	2025.884	0.116
45.010	2025.936	0.064
45.011	2025.965	0.035
45.011	2025.981	0.019
45.011	2025.989	0.011
45.011	2025.994	0.006
45.011	2025.997	0.003
45.011	2025.998	0.002
45.011	2025.999	0.001

- Starting at $w = 40$, with $\alpha = 0.005$.
- The whole run, all 23 rounds of it.
- From the □fth row of this block, w has stopped moving and the verdict is still shrinking.
- $\sqrt{2026} = 45.011110$, and no square root was ever taken.

α is the step size. The letter ε is reserved: it is the stopping tolerance, and at 11:00 it becomes the width of a di□erence window.

Letting the step shrink

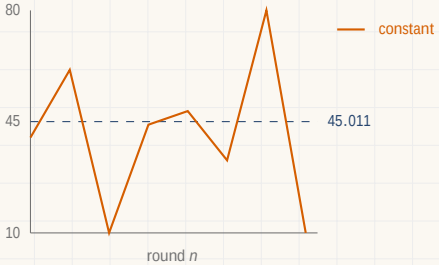
$w := w + \alpha_n \, l$, with $\alpha_n = 0.05/n$: the step gets smaller every round.



n	α_n	w	l

Letting the step shrink

$w := w + \alpha_n \, l$, with $\alpha_n = 0.05/n$: the step gets smaller every round.

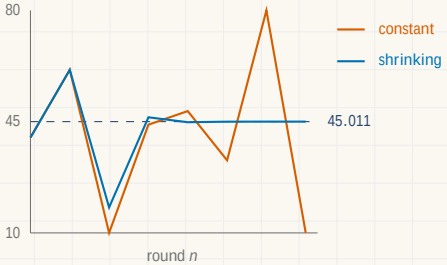


The constant run leaves the picture twice.

n	α_n	w	l

Letting the step shrink

$w := w + \alpha_n l$, with $\alpha_n = 0.05/n$: the step gets smaller every round.

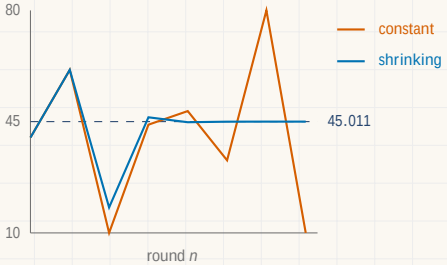


The constant run leaves the picture twice.

n	α_n	w	l
1	0.0500	40.000	$+426.000$
2	0.0250	61.300	-1731.690
3	0.0167	18.008	$+1701.721$
4	0.0125	46.370	-124.155

Letting the step shrink

$w := w + \alpha_n \, l$, with $\alpha_n = 0.05/n$: the step gets smaller every round.

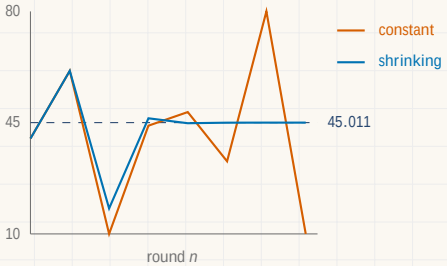


The constant run leaves the picture twice.

n	α_n	w	l
1	0.0500	40.000	+426.000
2	0.0250	61.300	−1731.690
3	0.0167	18.008	+1701.721
4	0.0125	46.370	−124.155
5	0.0100	44.818	+17.362
6	0.0083	44.991	+1.769

Letting the step shrink

$w := w + \alpha_n \, l$, with $\alpha_n = 0.05/n$: the step gets smaller every round.

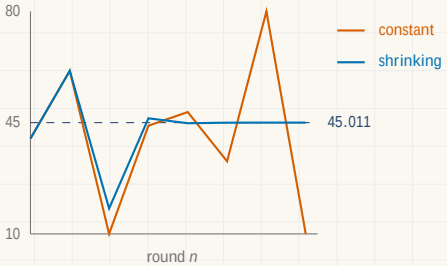


The constant run leaves the picture twice.

n	α_n	w	l
1	0.0500	40.000	+426.000
2	0.0250	61.300	−1731.690
3	0.0167	18.008	+1701.721
4	0.0125	46.370	−124.155
5	0.0100	44.818	+17.362
6	0.0083	44.991	+1.769
9	0.0056	45.010342	+0.069
12	0.0042	45.010985	+0.011

Letting the step shrink

$w := w + \alpha_n \, l$, with $\alpha_n = 0.05/n$: the step gets smaller every round.



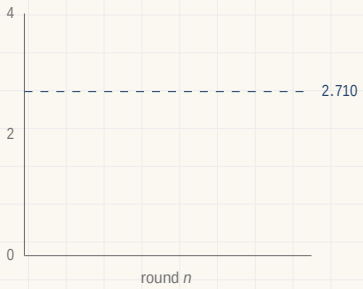
The constant run leaves the picture twice.

n	α_n	w	l
1	0.0500	40.000	$+426.000$
2	0.0250	61.300	-1731.690
3	0.0167	18.008	$+1701.721$
4	0.0125	46.370	-124.155
5	0.0100	44.818	$+17.362$
6	0.0083	44.991	$+1.769$
9	0.0056	45.010342	$+0.069$
12	0.0042	45.010985	$+0.011$

$\sqrt{2026} = 45.011110$. The digits arrive one at a time, and no square root was ever taken.

Letting the step shrink: the equation

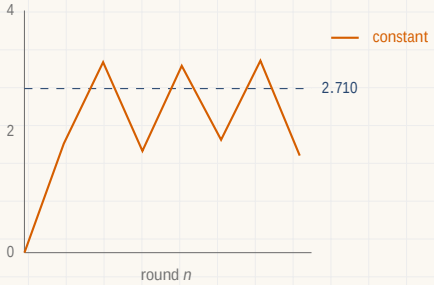
The same change, on $w^3 - 3w^2 + 3w - 2 = 4$, with $\alpha_n = 0.3/n$.



n	α_n	w	I

Letting the step shrink: the equation

The same change, on $w^3 - 3w^2 + 3w - 2 = 4$, with $\alpha_n = 0.3/n$.

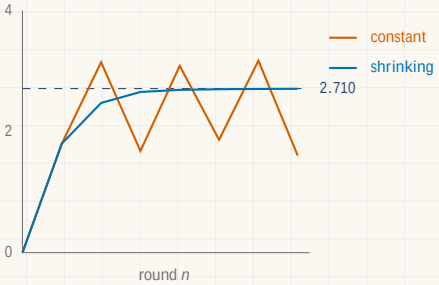


The constant run never lands.

n	α_n	w	l

Letting the step shrink: the equation

The same change, on $w^3 - 3w^2 + 3w - 2 = 4$, with $\alpha_n = 0.3/n$.

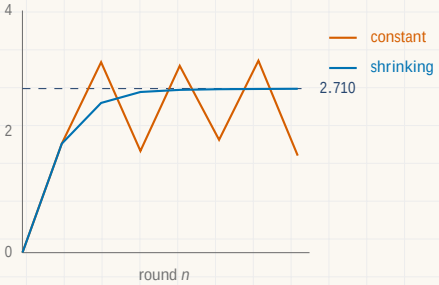


The constant run never lands.

n	α_n	w	l
1	0.3000	0.000	+6.000
2	0.1500	1.800	+4.488
3	0.1000	2.473	+1.803
4	0.0750	2.654	+0.480

Letting the step shrink: the equation

The same change, on $w^3 - 3w^2 + 3w - 2 = 4$, with $\alpha_n = 0.3/n$.

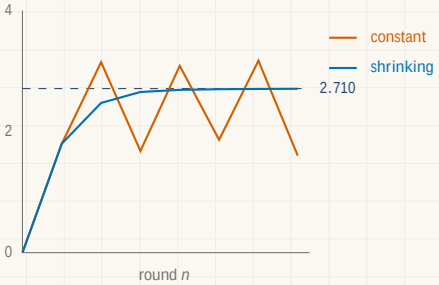


The constant run never lands.

n	α_n	w	l
1	0.3000	0.000	+6.000
2	0.1500	1.800	+4.488
3	0.1000	2.473	+1.803
4	0.0750	2.654	+0.480
5	0.0600	2.689	+0.178
6	0.0500	2.700	+0.086

Letting the step shrink: the equation

The same change, on $w^3 - 3w^2 + 3w - 2 = 4$, with $\alpha_n = 0.3/n$.

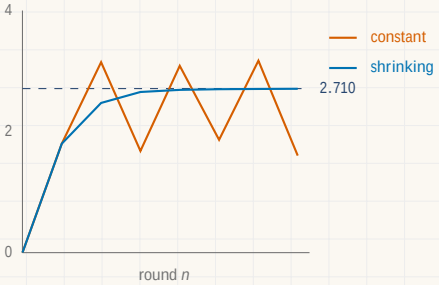


The constant run never lands.

n	α_n	w	l
1	0.3000	0.000	+6.000
2	0.1500	1.800	+4.488
3	0.1000	2.473	+1.803
4	0.0750	2.654	+0.480
5	0.0600	2.689	+0.178
6	0.0500	2.700	+0.086
9	0.0333	2.707640	+0.020
12	0.0250	2.709048	+0.008

Letting the step shrink: the equation

The same change, on $w^3 - 3w^2 + 3w - 2 = 4$, with $\alpha_n = 0.3/n$.



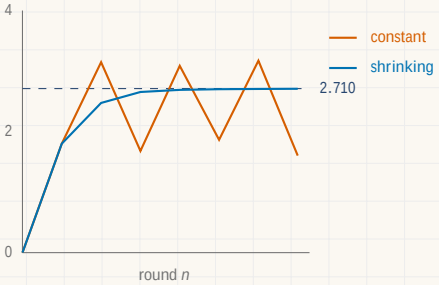
The constant run never lands.

n	α_n	w	l
1	0.3000	0.000	+6.000
2	0.1500	1.800	+4.488
3	0.1000	2.473	+1.803
4	0.0750	2.654	+0.480
5	0.0600	2.689	+0.178
6	0.0500	2.700	+0.086
9	0.0333	2.707640	+0.020
12	0.0250	2.709048	+0.008

The answer is $1 + \sqrt[3]{5} = 2.709976$.

Letting the step shrink: the equation

The same change, on $w^3 - 3w^2 + 3w - 2 = 4$, with $\alpha_n = 0.3/n$.



The constant run never lands.

n	α_n	w	f
1	0.3000	0.000	+6.000
2	0.1500	1.800	+4.488
3	0.1000	2.473	+1.803
4	0.0750	2.654	+0.480
5	0.0600	2.689	+0.178
6	0.0500	2.700	+0.086
9	0.0333	2.707640	+0.020
12	0.0250	2.709048	+0.008

The answer is $1 + \sqrt[3]{5} = 2.709976$.

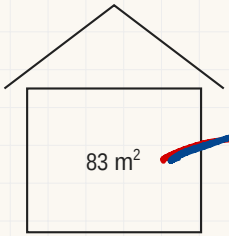
A step that never shrinks has to be lucky. A step that shrinks does not.

3

Section 3

Estimating house prices

One house, one measurement



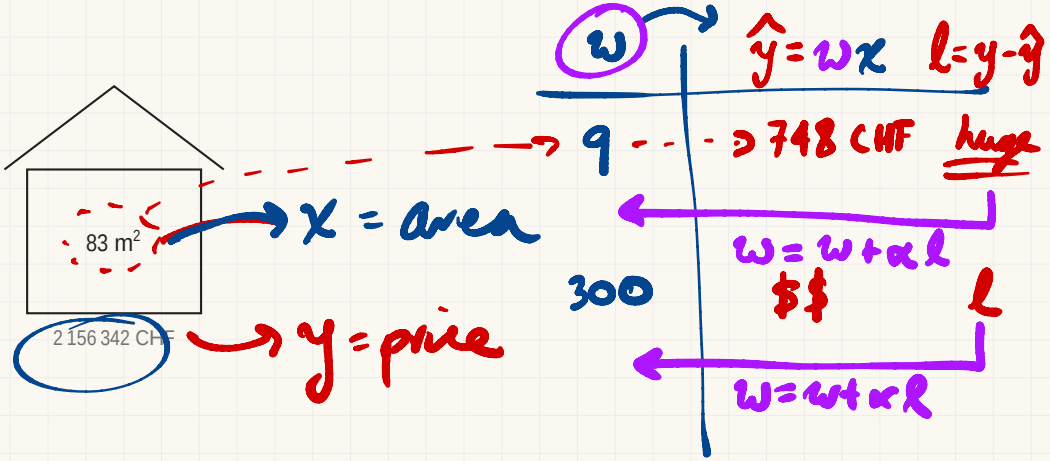
$ is prop to area

x = area

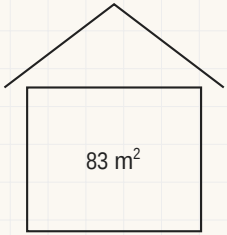
y = price

y = wx

One house, one measurement

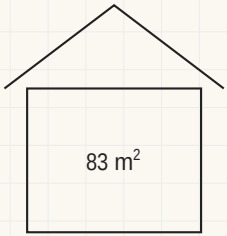


One house, one measurement



Data $x = 83$, the area. $y = 2\,156\,342$, the price.

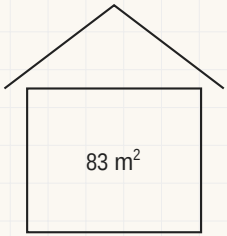
One house, one measurement



Data $x = 83$, the area. $y = 2\,156\,342$, the price.

Hypothesis Price is proportional to area, so the estimate is $\hat{y} = w\,x$.

One house, one measurement

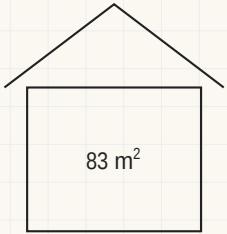


Data $x = 83$, the area. $y = 2\,156\,342$, the price.

Hypothesis Price is proportional to area, so the estimate is $\hat{y} = w\,x$.

Verdict The signed miss $l = y - \hat{y}$.

One house, one measurement



Data	$x = 83$, the area. $y = 2\,156\,342$, the price.
Hypothesis	Price is proportional to area, so the estimate is $\hat{y} = w\,x$.
Verdict	The signed miss $l = y - \hat{y}$.
Problem	Find the w that drives l to zero.

One row and one unknown, so the answer is $y/x = 25\,980.02$ francs per square metre, and the loop that follows never divides.

One house: the loop

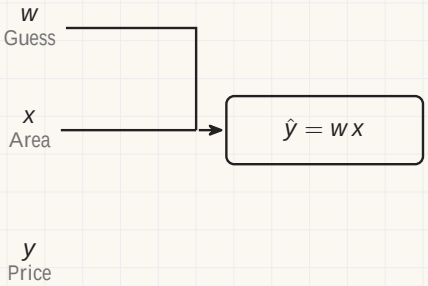
One house: the loop

w
Guess

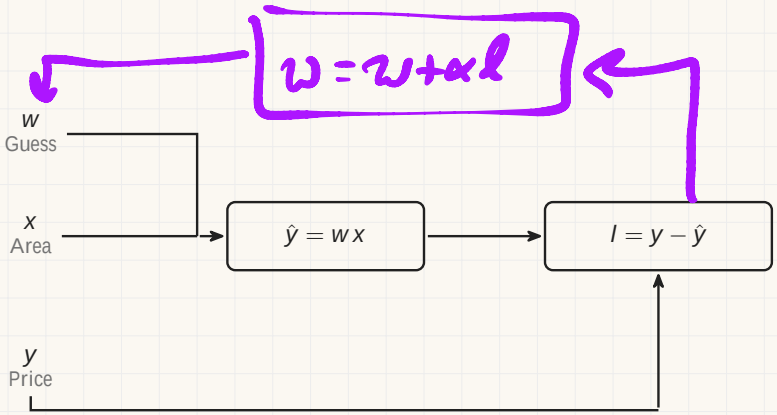
x
Area

y
Price

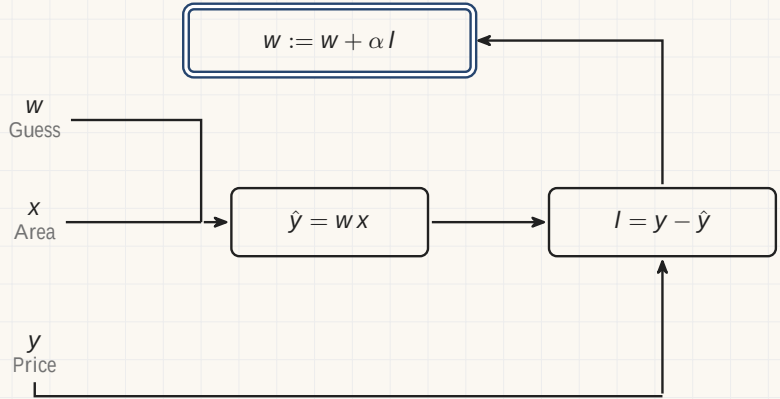
One house: the loop



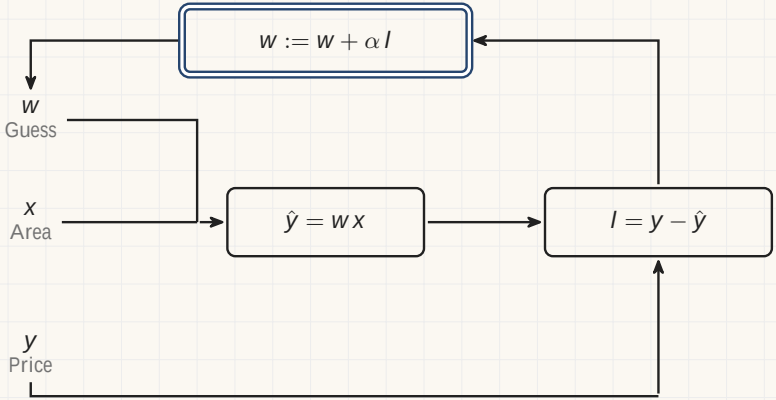
One house: the loop



One house: the loop



One house: the loop



One house: the algorithm

```
def fit(x, y, w, alpha):
    l = y - w * x              # the verdict: y minus the estimate
    while abs(l) > 0.5:        # more than half a franc out
        w = w + alpha * l      # the update
        l = y - w * x          # score the new guess
    return w
```

One house: the algorithm

```
def fit(x, y, w, alpha):
    l = y - w * x                # the verdict: y minus the estimate
    while abs(l) > 0.5:          # more than half a franc out
        w = w + alpha * l        # the update
        l = y - w * x            # score the new guess
    return w
```

On the house above:

```
fit(83, 2156342, 20000, 0.01)
```

One house: the algorithm

```
def fit(x, y, w, alpha):
    l = y - w * x                # the verdict: y minus the estimate
    while abs(l) > 0.5:          # more than half a franc out
        w = w + alpha * l        # the update
        l = y - w * x            # score the new guess
    return w
```

On the house above:

```
fit(83, 2156342, 20000, 0.01)
```

Starting from a guess of 20 000 francs per square metre, this returns 25 980.02 after 8 rounds.

Many houses, one measurement

House	Area	Price

Many houses, one measurement

House	Area	Price
1	83	2 156 342
2	62	1 461 635
3	104	2 650 166
4	128	3 184 899
5	145	3 651 113

House 1 is the house on the last slide. Houses 2 to 5 are synthetic, generated with a □xed seed.

Many houses, one measurement

House	Area	Price
1	83	2 156 342
2	62	1 461 635
3	104	2 650 166
4	128	3 184 899
5	145	3 651 113

House 1 is the house on the last slide. Houses 2 to 5 are synthetic, generated with a fixed seed.

Hypothesis The same one, row by row: $\hat{y}_i = w\,x_i$.

Many houses, one measurement

House	Area	Price
1	83	2 156 342
2	62	1 461 635
3	104	2 650 166
4	128	3 184 899
5	145	3 651 113

House 1 is the house on the last slide. Houses 2 to 5 are synthetic, generated with a fixed seed.

Hypothesis The same one, row by row: $\hat{y}_i = w\,x_i$.

Verdict One miss per row, $l_i = y_i - \hat{y}_i$.

Many houses, one measurement

House	Area	Price
1	83	2 156 342
2	62	1 461 635
3	104	2 650 166
4	128	3 184 899
5	145	3 651 113

House 1 is the house on the last slide. Houses 2 to 5 are synthetic, generated with a fixed seed.

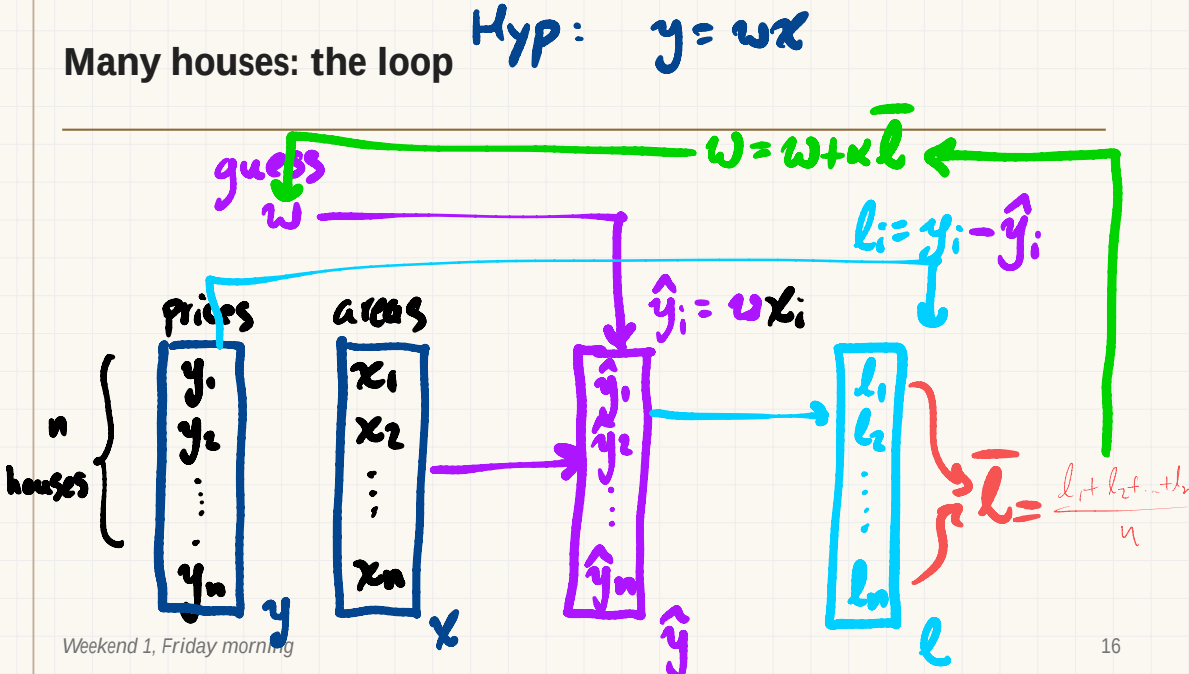
Hypothesis The same one, row by row: $\hat{y}_i = w\,x_i$.

Verdict One miss per row, $l_i = y_i - \hat{y}_i$.

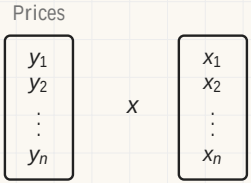
Problem No single w empties every row. Drive the *average* verdict $\bar{l}$ to zero instead.

$$\bar{l} = \frac{l_1 + l_2 + \cdots + l_n}{n}$$

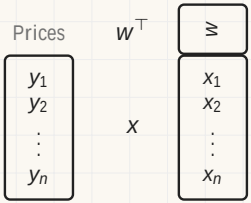
Many houses: the loop



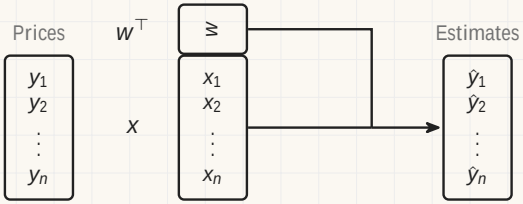
Many houses: the loop



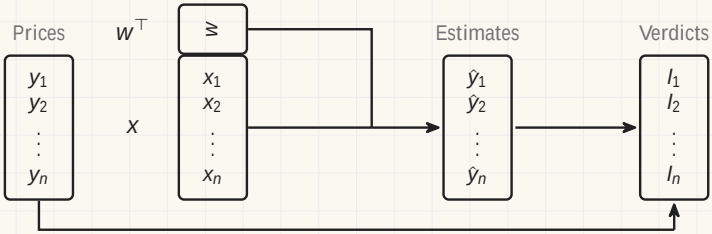
Many houses: the loop



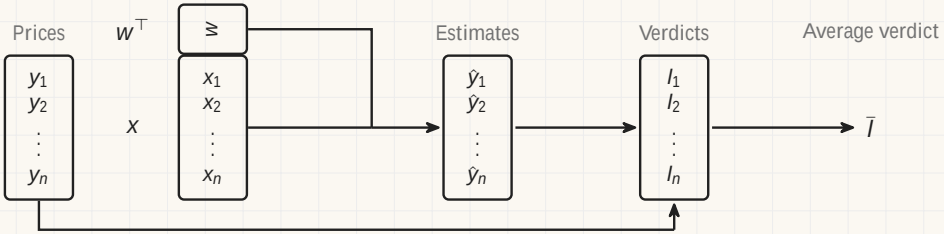
Many houses: the loop



Many houses: the loop

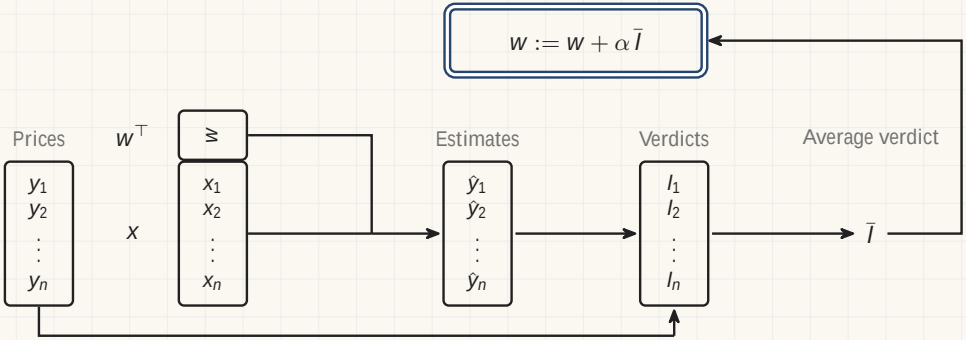


Many houses: the loop



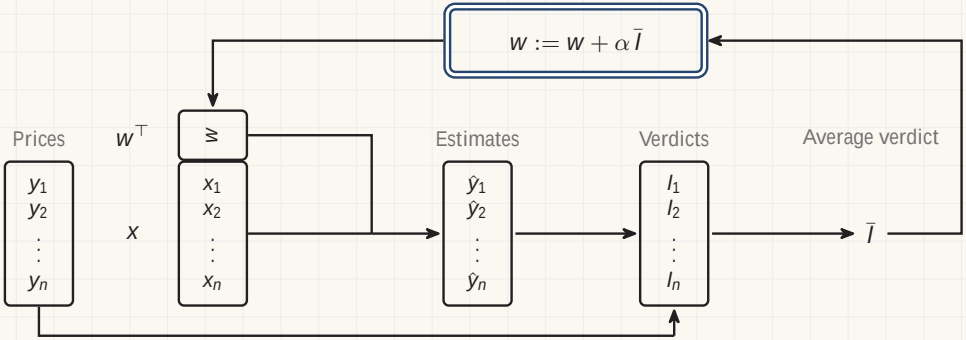
x holds the areas, and $\hat{y}_i = w\,x_i, \quad l_i = y_i - \hat{y}_i, \quad \bar{l} = (l_1 + \cdots + l_n)/n$

Many houses: the loop



x holds the areas, and $\hat{y}_i = w\,x_i, \quad l_i = y_i - \hat{y}_i, \quad \bar{l} = (l_1 + \cdots + l_n)/n$

Many houses: the loop



x holds the areas, and $\hat{y}_i = w\,x_i, \quad l_i = y_i - \hat{y}_i, \quad \bar{l} = (l_1 + \cdots + l_n)/n$

Many houses: the algorithm

```
def verdict(x, y, w):          # the average miss over the rows
    total = 0
    for i in range(len(x)):
        total = total + (y[i] - w * x[i])
    return total / len(x)
def fit(x, y, w, alpha):
    l_bar = verdict(x, y, w)
    while abs(l_bar) > 0.5:
        w = w + alpha * l_bar
        l_bar = verdict(x, y, w)
    return w
```

Many houses: the algorithm

```
def verdict(x, y, w):          # the average miss over the rows
    total = 0
    for i in range(len(x)):
        total = total + (y[i] - w * x[i])
    return total / len(x)
def fit(x, y, w, alpha):
    l_bar = verdict(x, y, w)
    while abs(l_bar) > 0.5:
        w = w + alpha * l_bar
        l_bar = verdict(x, y, w)
    return w
```

The estimate, the verdict and the update are unchanged. Only the averaging is new.

Many houses: the algorithm

```
def verdict(x, y, w):          # the average miss over the rows
    total = 0
    for i in range(len(x)):
        total = total + (y[i] - w * x[i])
    return total / len(x)
def fit(x, y, w, alpha):
    l_bar = verdict(x, y, w)
    while abs(l_bar) > 0.5:
        w = w + alpha * l_bar
        l_bar = verdict(x, y, w)
    return w
```

The estimate, the verdict and the update are unchanged. Only the averaging is new.

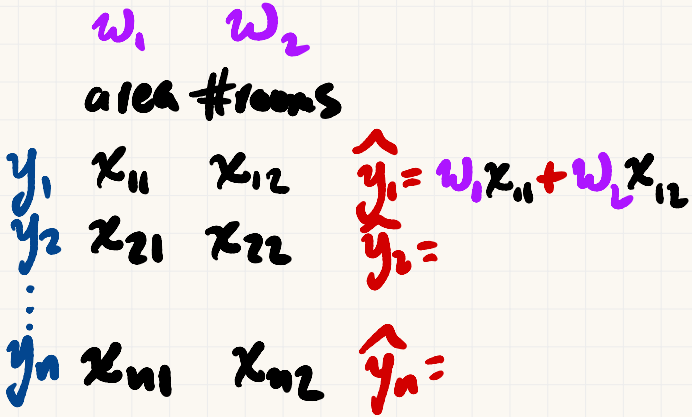
On the five houses, starting from 20 000 with $\alpha = 0.005$, this returns 25 103.74 after 19 rounds.

Two measurements for each house

House	Area	Rooms	Price

Two measurements for each house

House	Area	Rooms	Price
1	83	3.5	2 156 342
2	62	2.5	1 461 635
3	104	4.5	2 650 166
4	128	5.5	3 184 899
5	145	6.5	3 651 113



Two measurements for each house

House	Area	Rooms	Price
1	83	3.5	2 156 342
2	62	2.5	1 461 635
3	104	4.5	2 650 166
4	128	5.5	3 184 899
5	145	6.5	3 651 113

Hypothesis Each measurement gets its own weight: $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2}$.

Two measurements for each house

House	Area	Rooms	Price
1	83	3.5	2 156 342
2	62	2.5	1 461 635
3	104	4.5	2 650 166
4	128	5.5	3 184 899
5	145	6.5	3 651 113

Hypothesis Each measurement gets its own weight: $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2}$.

Verdict Still one number, $\bar{l}$, the average miss.

Two measurements for each house

House	Area	Rooms	Price
1	83	3.5	2 156 342
2	62	2.5	1 461 635
3	104	4.5	2 650 166
4	128	5.5	3 184 899
5	145	6.5	3 651 113

Hypothesis Each measurement gets its own weight: $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2}$.

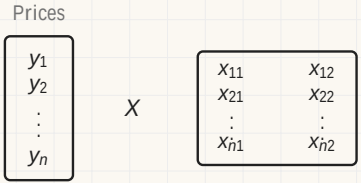
Verdict Still one number, $\bar{l}$, the average miss.

Problem Two unknowns, one verdict. It does not say which weight is at fault.

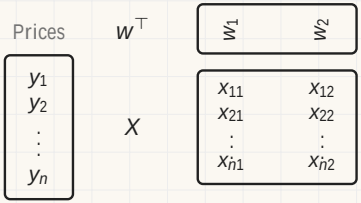
So move one weight at a time.

Two measurements: the loop

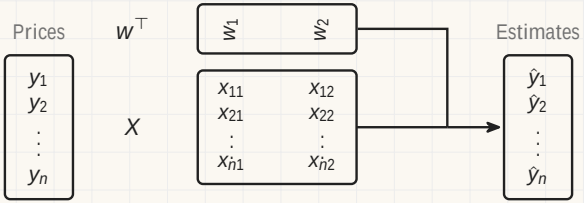
Two measurements: the loop



Two measurements: the loop

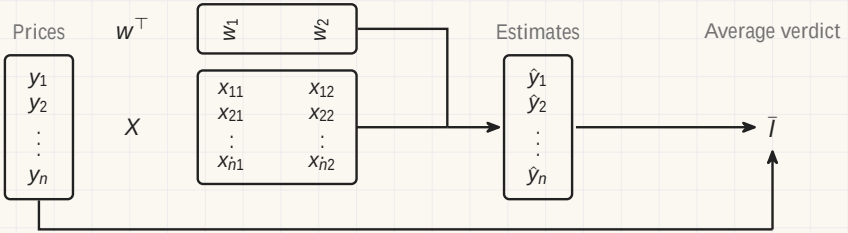


Two measurements: the loop



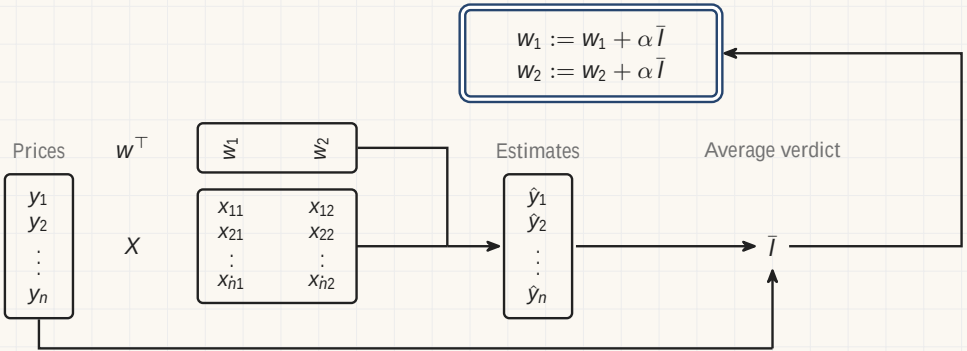
$\hat{y}_i = w_1 x_{i1} + w_2 x_{i2}$, and the columns of X are area and rooms.

Two measurements: the loop



$\hat{y}_i = w_1 x_{i1} + w_2 x_{i2}$, and the columns of X are area and rooms.

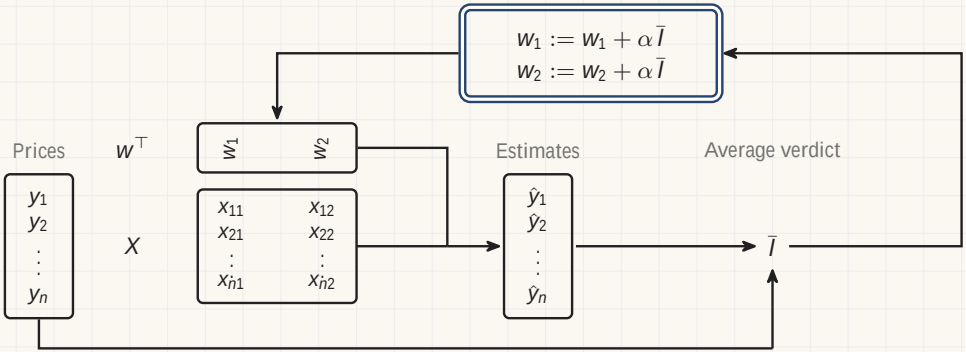
Two measurements: the loop



$\hat{y}_i = w_1 x_{i1} + w_2 x_{i2}$, and the columns of X are area and rooms.

One weight moves per round, and they take turns.

Two measurements: the loop



$\hat{y}_i = w_1 x_{i1} + w_2 x_{i2}$, and the columns of X are area and rooms.

One weight moves per round, and they take turns.

Two measurements: the algorithm

```
def verdict(x, y, w):
    total = 0
    for i in range(len(x)):
        total = total + (y[i] - w[0] * x[i][0] - w[1] * x[i][1])
    return total / len(x)

def fit(x, y, w, alpha):
    j = 0
    l_bar = verdict(x, y, w)
    while abs(l_bar) > 0.5:
        w[j] = w[j] + alpha * l_bar
        j = 1 - j                    # take turns
        l_bar = verdict(x, y, w)
    return w
```

Two measurements: the algorithm

```
def verdict(x, y, w):
    total = 0
    for i in range(len(x)):
        total = total + (y[i] - w[0] * x[i][0] - w[1] * x[i][1])
    return total / len(x)

def fit(x, y, w, alpha):
    j = 0
    l_bar = verdict(x, y, w)
    while abs(l_bar) > 0.5:
        w[j] = w[j] + alpha * l_bar
        j = 1 - j                    # take turns
        l_bar = verdict(x, y, w)
    return w
```

Starting from $w = (20\,000,\ 0)$ with $\alpha = 0.005$, this returns $(25\,000.71,\ 2\,390.34)$ after 37 rounds.

Many measurements for each house

House	Area	Rooms	Age	Walk	Price

Many measurements for each house

House	Area	Rooms	Age	Walk	Price
1	83	3.5	12	4	2 156 342
2	62	2.5	41	11	1 461 635
3	104	4.5	28	8	2 650 166
4	128	5.5	6	15	3 184 899
5	145	6.5	55	6	3 651 113

Many measurements for each house

House	Area	Rooms	Age	Walk	Price
1	83	3.5	12	4	2 156 342
2	62	2.5	41	11	1 461 635
3	104	4.5	28	8	2 650 166
4	128	5.5	6	15	3 184 899
5	145	6.5	55	6	3 651 113

- Each house is now a list of d measurements, $x_i = (x_{i1}, x_{i2}, \ldots, x_{id})$.

Many measurements for each house

House	Area	Rooms	Age	Walk	Price
1	83	3.5	12	4	2 156 342
2	62	2.5	41	11	1 461 635
3	104	4.5	28	8	2 650 166
4	128	5.5	6	15	3 184 899
5	145	6.5	55	6	3 651 113

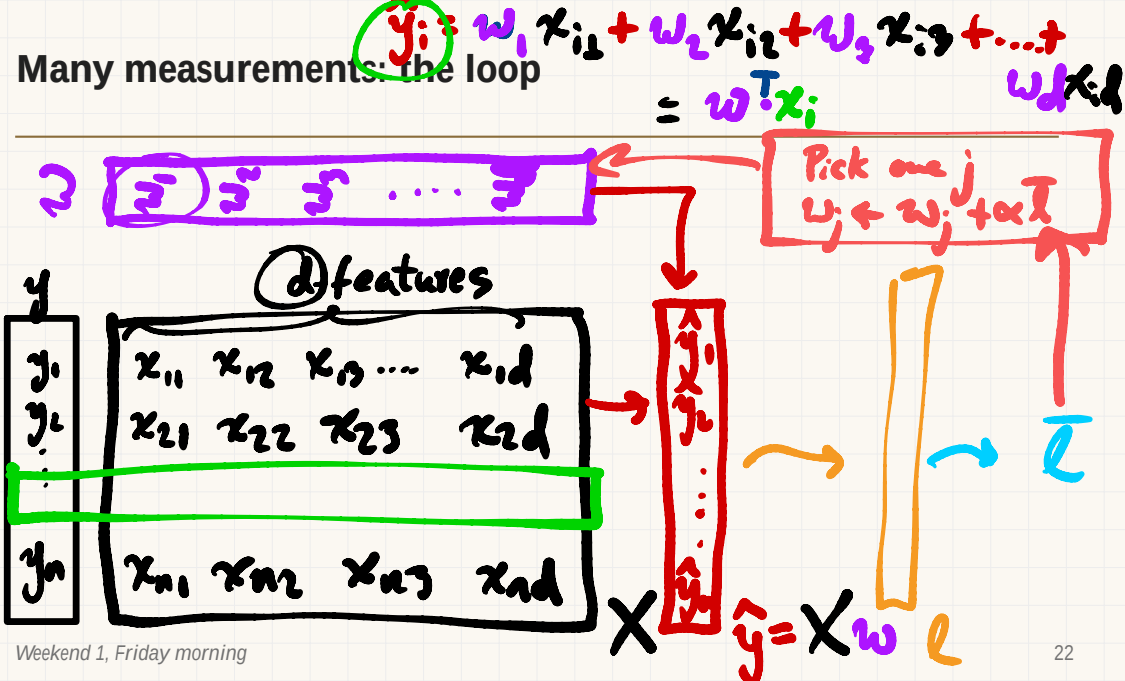
- Each house is now a list of d measurements, $x_i = (x_{i1}, x_{i2}, \ldots, x_{id})$.
- Each measurement keeps its own weight, so $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id}$.

Many measurements for each house

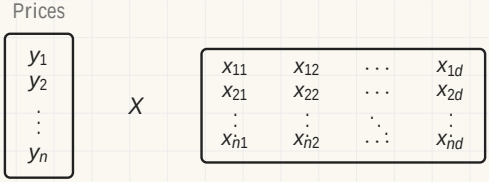
House	Area	Rooms	Age	Walk	Price
1	83	3.5	12	4	2 156 342
2	62	2.5	41	11	1 461 635
3	104	4.5	28	8	2 650 166
4	128	5.5	6	15	3 184 899
5	145	6.5	55	6	3 651 113

- Each house is now a list of d measurements, $x_i = (x_{i1}, x_{i2}, \ldots, x_{id})$.
- Each measurement keeps its own weight, so $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id}$.
- One verdict, d unknowns. Pick one weight per round and move it.

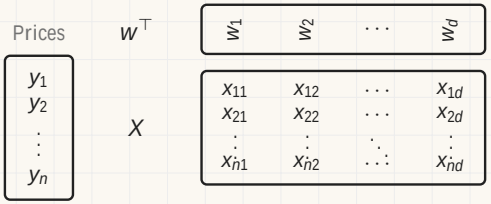
Many measurements: the loop



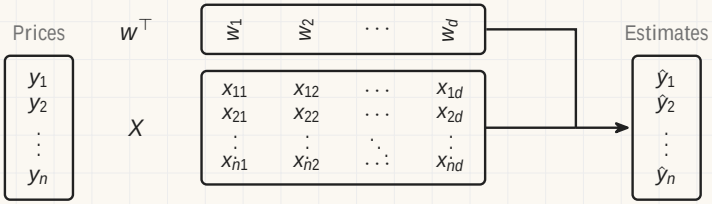
Many measurements: the loop



Many measurements: the loop

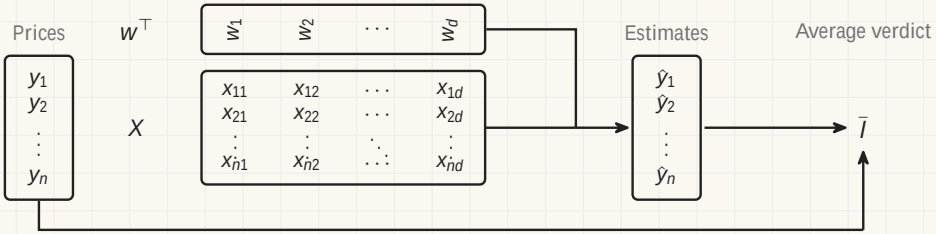


Many measurements: the loop



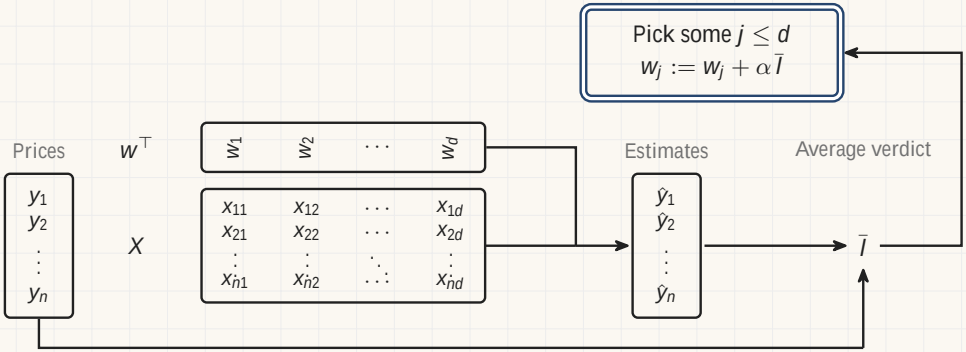
X has one row per house and one column per measurement, and $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id}$.

Many measurements: the loop



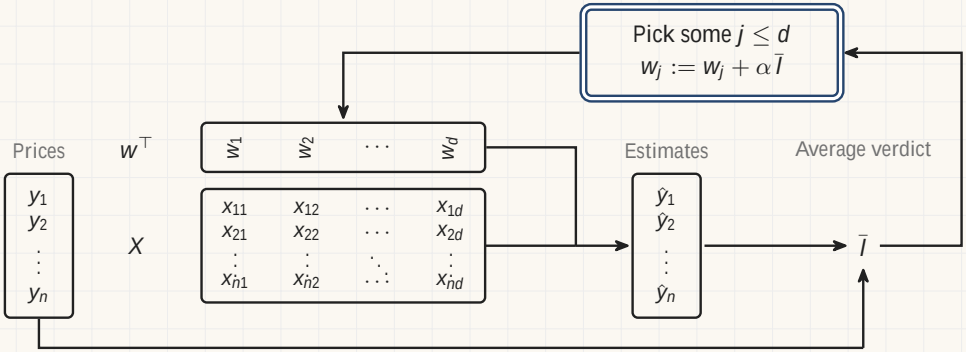
X has one row per house and one column per measurement, and $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id}$.

Many measurements: the loop



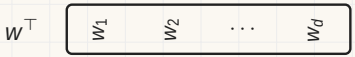
X has one row per house and one column per measurement, and $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id}$.

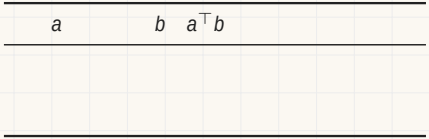
Many measurements: the loop



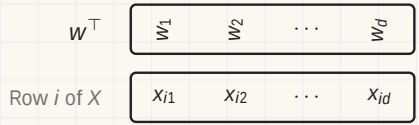
X has one row per house and one column per measurement, and $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id}$.

The inner product



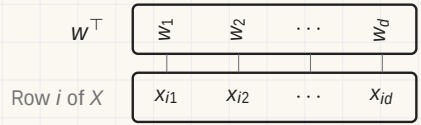


The inner product



a	b	$a^\top b$

The inner product

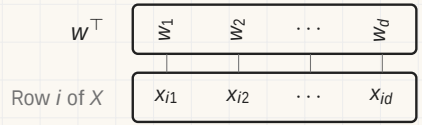


$$w^\top x_i = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id}$$

Multiply down each column, then add.

a	b	$a^\top b$

The inner product



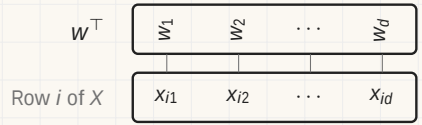
$$w^\top x_i = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id}$$

Multiply down each column, then add.

```
def inner(w, x):
    total = 0
    for j in range(len(w)):
        total = total + w[j] * x[j]
    return total
```

a	b	$a^\top b$

The inner product



$$w^\top x_i = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id}$$

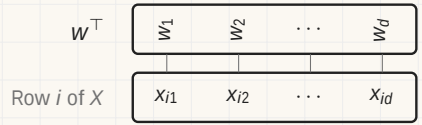
Multiply down each column, then add.

```
def inner(w, x):
    total = 0
    for j in range(len(w)):
        total = total + w[j] * x[j]
    return total
```

It also measures similarity.

a	b	$a^\top b$	
$(2, 1)$	$(2, 1)$	$+5$	The same direction

The inner product



$$w^\top x_i = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id}$$

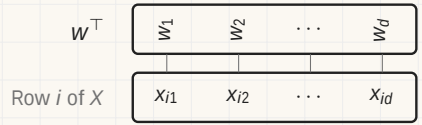
Multiply down each column, then add.

```
def inner(w, x):
    total = 0
    for j in range(len(w)):
        total = total + w[j] * x[j]
    return total
```

It also measures similarity.

a	b	$a^\top b$	
$(2, 1)$	$(2, 1)$	$+5$	The same direction
$(2, 1)$	$(-1, 2)$	0	At right angles
$(2, 1)$	$(-2, -1)$	-5	Opposite directions

The inner product



$$w^\top x_i = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id}$$

Multiply down each column, then add.

```
def inner(w, x):
    total = 0
    for j in range(len(w)):
        total = total + w[j] * x[j]
    return total
```

It also measures similarity.

a	b	$a^\top b$	
$(2, 1)$	$(2, 1)$	$+5$	The same direction
$(2, 1)$	$(-1, 2)$	0	At right angles
$(2, 1)$	$(-2, -1)$	-5	Opposite directions

It scores a document against a query, and one word against another in a language model. It returns all module.

Many measurements: the algorithm

```
def verdict(x, y, w):
    total = 0
    for i in range(len(x)):
        total = total + (y[i] - inner(w, x[i]))
    return total / len(x)
def fit(x, y, w, alpha):
    l_bar = verdict(x, y, w)
    while abs(l_bar) > 0.5:
        j = random.randrange(len(w))   # pick some j
        w[j] = w[j] + alpha * l_bar
        l_bar = verdict(x, y, w)
    return w
```

Many measurements: the algorithm

```
def verdict(x, y, w):
    total = 0
    for i in range(len(x)):
        total = total + (y[i] - inner(w, x[i]))
    return total / len(x)
def fit(x, y, w, alpha):
    l_bar = verdict(x, y, w)
    while abs(l_bar) > 0.5:
        j = random.randrange(len(w))   # pick some j
        w[j] = w[j] + alpha * l_bar
        l_bar = verdict(x, y, w)
    return w
```

Starting from $w = (20\,000,\ 0,\ 0,\ 0)$ with $\alpha = 0.005$, this returns $(24\,566.95,\ 976.25,\ 1\,351.85,\ 1\,506.32)$ after 51 rounds.

Many measurements: the algorithm

```
def verdict(x, y, w):
    total = 0
    for i in range(len(x)):
        total = total + (y[i] - inner(w, x[i]))
    return total / len(x)
def fit(x, y, w, alpha):
    l_bar = verdict(x, y, w)
    while abs(l_bar) > 0.5:
        j = random.randrange(len(w))   # pick some j
        w[j] = w[j] + alpha * l_bar
        l_bar = verdict(x, y, w)
    return w
```

Starting from $w = (20\,000,\ 0,\ 0,\ 0)$ with $\alpha = 0.005$, this returns $(24\,566.95,\ 976.25,\ 1\,351.85,\ 1\,506.32)$ after 51 rounds.

The three algorithms before it are this one with $d = 1$ and $n = 1$, then $d = 1$, then $d = 2$.

4

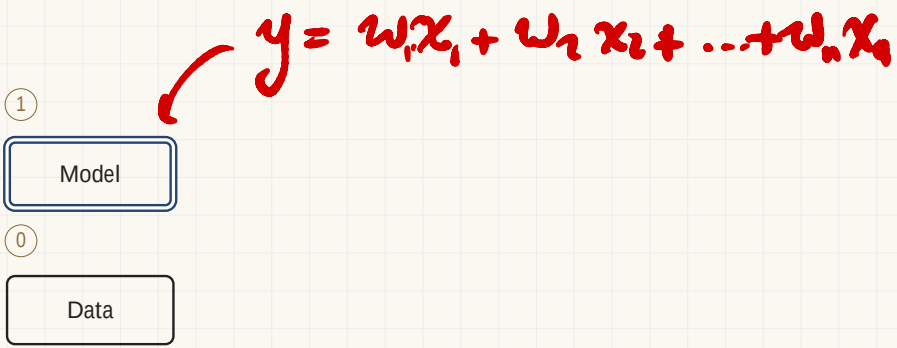
Section 4

In AI

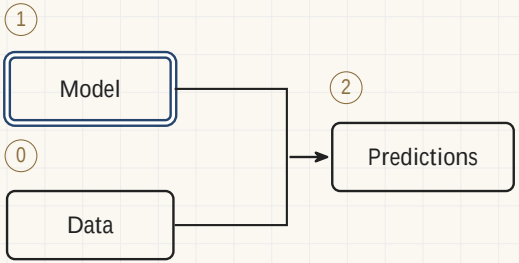
The training loop

The training loop

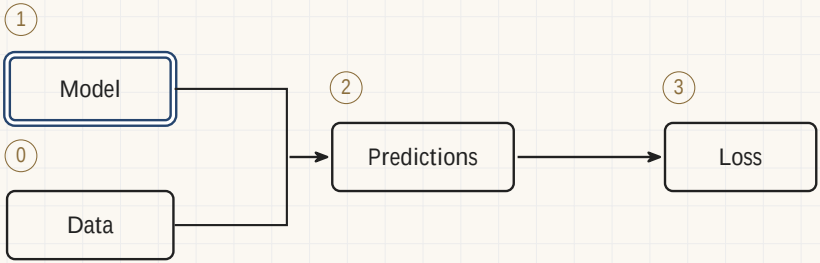
The training loop



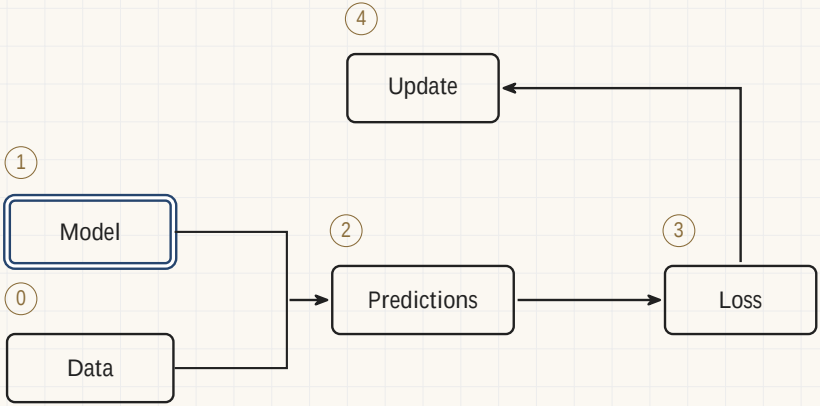
The training loop



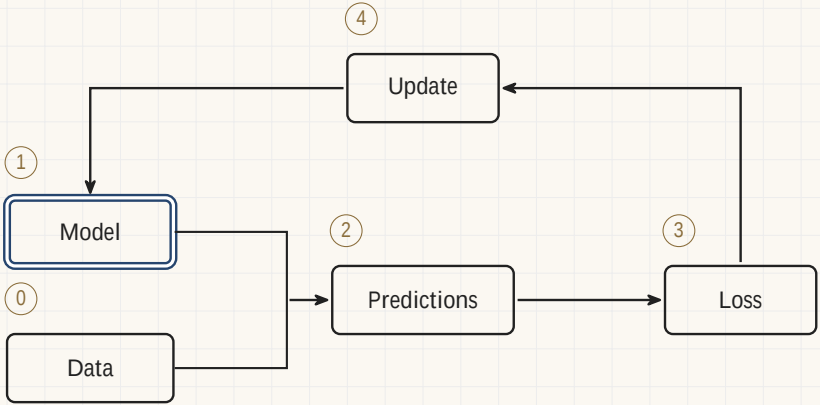
The training loop



The training loop



The training loop



Four boxes and a return arrow, run a few million times.

The morning’s four words, and their AI names

The morning’s four words, and their AI names

Guess → Model

The weights *w*

The morning’s four words, and their AI names

Guess	→	**Model** The weights *w*
Estimate	→	**Predictions** What the weights say about the data

The morning’s four words, and their AI names

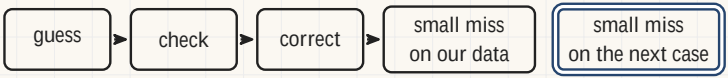
Guess	→	**Model** The weights *w*
Estimate	→	**Predictions** What the weights say about the data
Verdict	→	**Loss** How wrong, and in which direction

The morning's four words, and their AI names

Guess	→	**Model** The weights w
Estimate	→	**Predictions** What the weights say about the data
Verdict	→	**Loss** How wrong, and in which direction
Update	→	**Training step** $w := w + \alpha l$

Statistical learning theory

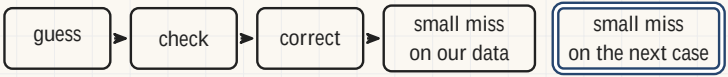
Vladimir Vapnik



Statistical learning theory

Vladimir Vapnik

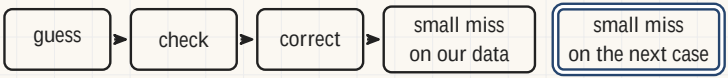
- All morning we moved *w* until the miss on *our* rows was small.



Statistical learning theory

Vladimir Vapnik

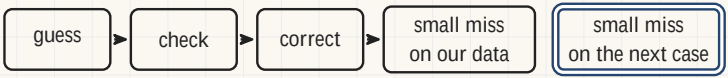
- All morning we moved *w* until the miss on *our* rows was small.
- Nobody pays for the miss on rows we already have.



Statistical learning theory

Vladimir Vapnik

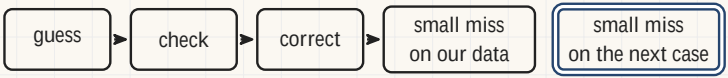
- All morning we moved *w* until the miss on *our* rows was small.
- Nobody pays for the miss on rows we already have.
- The question changes: when is a rule □tted on a sample entitled to hold o□ it?



Statistical learning theory

Vladimir Vapnik

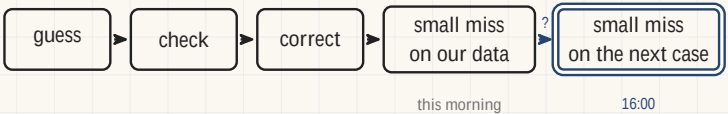
- All morning we moved *w* until the miss on *our* rows was small.
- Nobody pays for the miss on rows we already have.
- The question changes: when is a rule □tted on a sample entitled to hold o□ it?
- Vapnik's answer turns on how much a family of rules can bend. A family that can □t anything explains nothing.



Statistical learning theory

Vladimir Vapnik

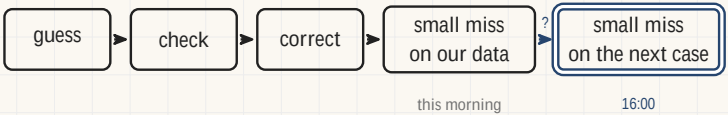
- All morning we moved *w* until the miss on *our* rows was small.
- Nobody pays for the miss on rows we already have.
- The question changes: when is a rule □tted on a sample entitled to hold o□ it?
- Vapnik's answer turns on how much a family of rules can bend. A family that can □t anything explains nothing.



Statistical learning theory

Vladimir Vapnik

- All morning we moved *w* until the miss on *our* rows was small.
- Nobody pays for the miss on rows we already have.
- The question changes: when is a rule □tted on a sample entitled to hold o□ it?
- Vapnik’s answer turns on how much a family of rules can bend. A family that can □t anything explains nothing.



Vapnik prices that arrow: the miss on the next case is at most the miss on our data, plus a penalty that grows with how much the family can bend and shrinks as rows are added. A price rather than a promise.

5

Section 5

Where this ends up

Modern applications

Most modern machine learning and AI applications are □rst-order approximation algorithms.

Family	What its verdict compares

Modern applications

Most modern machine learning and AI applications are □rst-order approximation algorithms.

Family	What its verdict compares
Large language models	The next token against the token that followed

Modern applications

Most modern machine learning and AI applications are □rst-order approximation algorithms.

Family	What its verdict compares
Large language models	The next token against the token that followed
Convolutional networks	The predicted label against the recorded label

Modern applications

Most modern machine learning and AI applications are □rst-order approximation algorithms.

Family	What its verdict compares
Large language models	The next token against the token that followed
Convolutional networks	The predicted label against the recorded label
Reasoning models	The □nished answer against a graded outcome

Modern applications

Most modern machine learning and AI applications are □rst-order approximation algorithms.

Family	What its verdict compares
Large language models	The next token against the token that followed
Convolutional networks	The predicted label against the recorded label
Reasoning models	The □nished answer against a graded outcome
Di□usion models	The predicted noise against the noise that was added

Modern applications

Most modern machine learning and AI applications are □rst-order approximation algorithms.

Family	What its verdict compares
Large language models	The next token against the token that followed
Convolutional networks	The predicted label against the recorded label
Reasoning models	The □nished answer against a graded outcome
Di□usion models	The predicted noise against the noise that was added

Four di□erent verdicts. One update rule, and it is the one on the square-root table.